

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2025 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Model 2025	2
SUBIECTUL I (30 de puncte)	2
1. Structura de date: Arbori	2
2. Ergonomia postului de lucru	2
SUBIECTUL al II-lea (30 de puncte)	2
1. Programare: Lanț muntos (Creastă)	2
2. Algoritm eficient: Tije discuri	4
SUBIECTUL al III-lea (30 de puncte)	5
1. Proiectarea unei strategii didactice: Formatarea textului (Secvența B)	5
2. Evaluare: Itemi obiectivi (Alegere multiplă)	5
II. Rezolvare Titularizare Varianta 3 (15 iulie 2025)	6
SUBIECTUL I	6
1. Structura de date: Coadă	6
2. Arhitectura sistemelor de calcul: UCP	6
SUBIECTUL al II-lea	6
1. Programare: Combinări bancnote (Backtracking)	6
2. Algoritmi eficienți: Suprapunere programări cabinet medic	6
SUBIECTUL al III-lea	6
1. Proiectare Test de Evaluare și Barem	6
2. Strategie didactică: Animații și Modele 3D	6

I. Rezolvare Titularizare Model 2025

SUBIECTUL I (30 de puncte)

1. Structura de date: Arbori

- **Graf neorientat:** Pereche ordonată $G = (V, E)$, unde V este mulțimea nodurilor și E mulțimea muchiilor.
- **Lanț:** Succesiune de noduri interconectate prin muchii distincte.
- **Ciclu:** Un lanț simplu în care nodul de pornire coincide cu cel de sosire.
- **Proprietăți arbore:**
 1. Conex și fără cicluri.
 2. Fără cicluri și are $N - 1$ muchii.
 3. Conex și are $N - 1$ muchii.
 4. Între oricare două noduri există un lanț simplu unic.
 5. Dacă se elimină o muchie, devine neconex; dacă se adaugă o muchie, se formează exact un ciclu.

• Problemă (Parcurgerea în lățime a unui arbore):

► C++:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
vector<int> adj[100];
bool viz[100];
void BFS(int start) {
    queue<int> Q;
    Q.push(start);
    viz[start] = true;
    while(!Q.empty()) {
        int u = Q.front(); Q.pop();
        cout << u << " ";
        for(int v : adj[u])
            if(!viz[v]) { viz[v] = true; Q.push(v); }
    }
}
```

► Pascal:

```
// Parcurgerea în lățime cu coadă pe tablou de adiacență.
```

2. Ergonomia postului de lucru

• Dispozitive periferice cu impact asupra sănătății:

1. **Monitorul:** Poate cauza oboseală oculară (astenopie) sau dureri de cap. Recomandare: Distanța ecran-ochi de 50-70 cm și utilizarea filtrelor de lumină albastră.
2. **Tastatura/Mouse-ul:** Pot genera sindromul de tunel carpian. Recomandare: Utilizarea suporturilor ergonomice pentru încheieturi.

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Lanț muntos (Creastă)

Soluție C++:

```

#include <iostream>
#include <cmath>
using namespace std;

int varf(int n, int a[50][50], int k) {
    int r = k - 1;
    int max_val = a[r][0];
    int max_pos = 0;
    for (int j = 1; j < n; ++j) {
        if (a[r][j] > max_val) {
            max_val = a[r][j];
            max_pos = j;
        }
    }
    for (int j = 0; j < n; ++j) {
        if (j != max_pos && a[r][j] == max_val) return 0;
    }
    for (int j = 0; j < max_pos; ++j) {
        if (a[r][j] >= a[r][j+1]) return 0;
    }
    for (int j = max_pos; j < n - 1; ++j) {
        if (a[r][j] <= a[r][j+1]) return 0;
    }
    return max_pos + 1;
}

int main() {
    int n, a[50][50];
    if (!(cin >> n)) return 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> a[i][j];

    bool e_creasta = true;
    int prev_p = varf(n, a, 1);
    if (prev_p == 0) e_creasta = false;

    for (int k = 2; k <= n && e_creasta; ++k) {
        int curr_p = varf(n, a, k);
        if (curr_p == 0 || abs(curr_p - prev_p) > 1) {
            e_creasta = false;
        }
        prev_p = curr_p;
    }

    if (e_creasta) cout << "DA" << endl;
    else cout << "NU" << endl;

    return 0;
}

```

Soluție Pascal:

```

program LancMuntos;
var
    n, i, j, prev_p, curr_p: integer;
    a: array[1..50, 1..50] of integer;

```

```

e_creasta: boolean;

function varf(n: integer; var a: array of integer; k: integer): integer;
var
    r, max_val, max_pos, col: integer;
    is_mountain: boolean;
begin
    r := k - 1;
    max_val := a[r * n + 0];
    max_pos := 0;
    for col := 1 to n - 1 do
    begin
        if a[r * n + col] > max_val then
        begin
            max_val := a[r * n + col];
            max_pos := col;
        end;
    end;

    is_mountain := true;
    for col := 0 to n - 1 do
        if (col <> max_pos) and (a[r * n + col] = max_val) then is_mountain := false;

    for col := 0 to max_pos - 1 do
        if a[r * n + col] >= a[r * n + col + 1] then is_mountain := false;

    for col := max_pos to n - 2 do
        if a[r * n + col] <= a[r * n + col + 1] then is_mountain := false;

    if is_mountain then varf := max_pos + 1
    else varf := 0;
end;
// ... (cod complet pentru citire și verificare)
begin
end.

```

2. Algoritm eficient: Tije discuri

- **Algoritmul:** Rămâne o listă a diametrelor din vârful fiecărei tije ocupate. Această listă este întotdeauna sortată crescător. Căutăm binar prima tijă adecvată. Complexitate: $O(N \log N)$ timp și $O(N)$ spațiu.

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu2025.txt");
    if (!fin) return 1;
    vector<int> R;
    int d;
    while (fin >> d) {
        auto it = lower_bound(R.begin(), R.end(), d);
    }
}

```

```

        if (it == R.end()) R.push_back(d);
        else *it = d;
    }
    fin.close();
    cout << R.size() << endl;
    return 0;
}

```

Soluție Pascal:

```

program TijeDiscuri;
var
    fin: text;
    R: array[1..100000] of integer;
    num_rods, d, pos, st, dr, mid: integer;
begin
    assign(fin, 'titu2025.txt'); reset(fin);
    num_rods := 0;
    while not eof(fin) do
        begin
            read(fin, d);
            st := 1; dr := num_rods; pos := -1;
            while st <= dr do
                begin
                    mid := (st + dr) div 2;
                    if R[mid] >= d then
                        begin
                            pos := mid; dr := mid - 1;
                        end
                    else st := mid + 1;
                end;
            if pos = -1 then
                begin
                    num_rods := num_rods + 1;
                    R[num_rods] := d;
                end
            else R[pos] := d;
            end;
        close(fin);
        writeln(num_rods);
    end.

```

SUBIECTUL al III-lea (30 de puncte)

1. Proiectarea unei strategii didactice: Formatarea textului (Secvența B)

- **Tip lecție:** Lecție de formare a priceperilor și deprinderilor practice.
- **Strategia:** Elevii primesc un text neformatat. Folosind instrucțiunile profesorului, aplică pe rând stiluri (bold, italic, subliniat), mărin și micșorând fontul pentru a crea un afiș publicitar.

2. Evaluare: Itemi obiectivi (Alegere multiplă)

- **Item pentru Backtracking (Secvența A):** Care este structura spațiului stărilor pentru generarea permutărilor? A. Produs cartezian. | B. Vector de lungime constantă. | C. Arbore de căutare. | D. Graf conex. Răspuns: C.

II. Rezolvare Titularizare Varianta 3 (15 iulie 2025)

[Subiect PDF](file:

(Notă: Subiectul este identic din punct de vedere științific cu cel prezentat în modelul 2026).

SUBIECTUL I

1. Structura de date: Coadă

Identic cu rezolvarea oferită pentru Titularizare 2026 (Secțiunea I.1). Prezintă accesul FIFO, cele 2 reprezentări (secvențială și înlănțuită) și implementarea algoritmului Lee.

2. Arhitectura sistemelor de calcul: UCP

Identic cu rezolvarea de la Titularizare 2026 (Secțiunea I.2).

SUBIECTUL al II-lea

1. Programare: Combinări bancnote (Backtracking)

Codul C++ și Pascal este identic cu cel din modelul 2026 (Secțiunea II.1).

2. Algoritmi eficienți: Suprapunere programări cabinet medic

Codul C++ și Pascal este identic cu cel din modelul 2026 (Secțiunea II.2).

SUBIECTUL al III-lea

1. Proiectare Test de Evaluare și Barem

Identic cu testul de la Titularizare 2026 (Secțiunea III.1).

2. Strategie didactică: Animații și Modele 3D

Identic cu abordarea didactică propusă pentru Titularizare 2026 (Secțiunea III.2).